Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

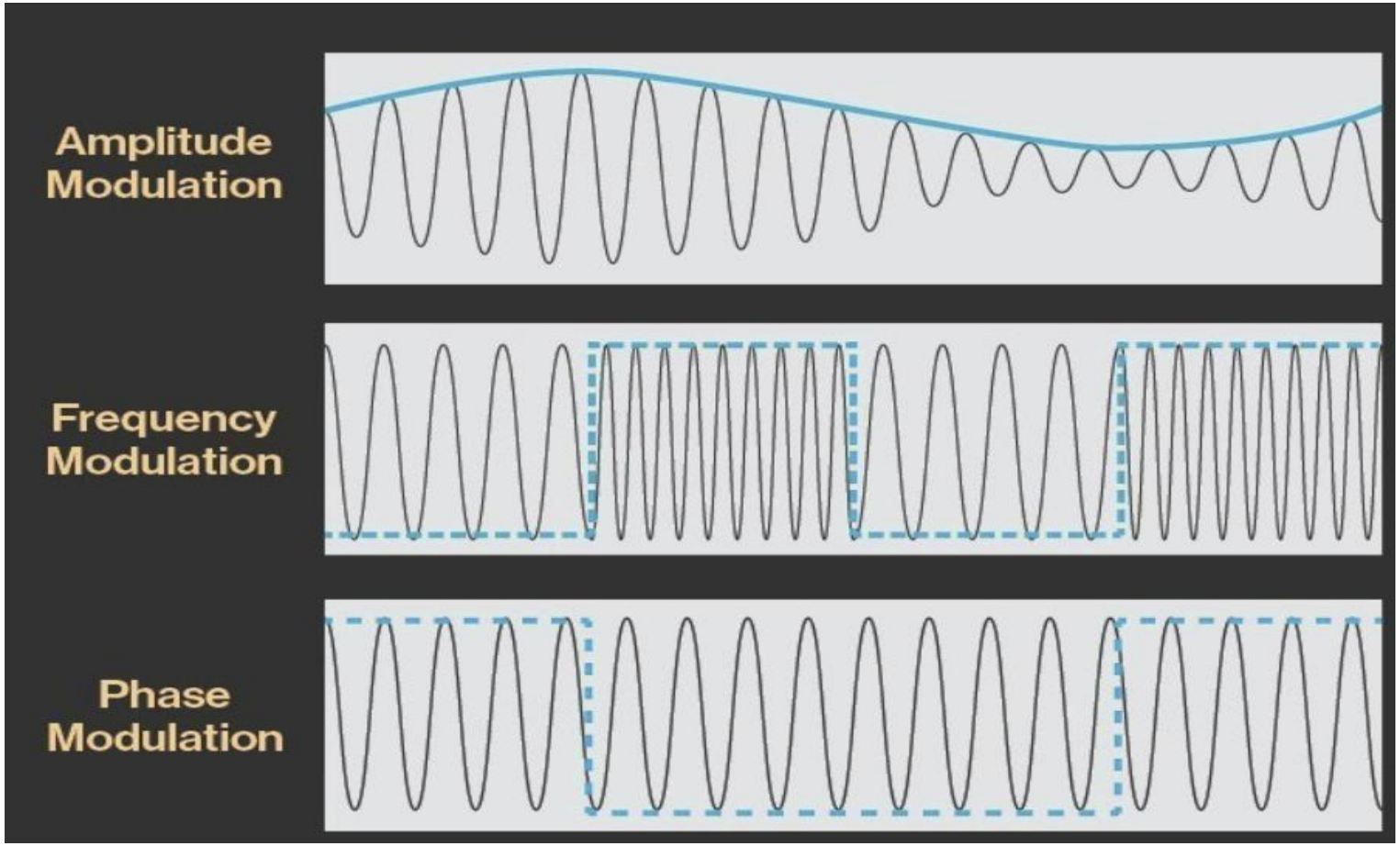
- Differential Coding of Audio
- Lossless Predictive Coding
- DPCM
- Lossless compression algorithms
- Basics of Information Theory
- Run length coding
- Variable length coding
- Shanon Fano algorithm
- Huffman coding
- Adaptive Huffman coding
- LZW

Modulation

- In electronics and telecommunications, **modulation** is the process of varying one or more properties of a periodic waveform, called the *carrier signal*, with a separate signal called the *modulation signal* that typically contains information to be transmitted.

- **Amplitude modulation (AM):** The height of the signal carrier is varied to represent the data being added to the signal.
- **Frequency modulation (FM):** The frequency of the carrier waveform is varied to reflect the frequency of the data.
- **Phase modulation (PM):** The phase of the carrier waveform is varied to reflect changes in the frequency of the data. In PM, the frequency is unchanged while the phase is changed relative to the base carrier frequency. It is similar to FM.

Modulation



Modulation is the process of encoding information in a transmitted signal, while demodulation is the process of extracting information from the transmitted signal

PCM in general

- We say that the boundaries for quantizer input intervals that will all be mapped into the same output level form a coder mapping, and the representative values that are the output values from a quantizer are a decoder mapping.
- Since we quantize, we may choose to create either an accurate or less accurate representation of sound magnitude values. Finally, we may wish to compress the data, by assigning a bitstream that uses fewer bits for the most prevalent signal values.

PCM in general

- Every compression scheme has three stages:
- 1. Transformation. The input data is transformed to a new representation that is easier or more efficient to compress. For example, in Predictive Coding, we predict the next signal from previous ones and transmit the prediction error.
- 2. Loss. We may introduce loss of information. Quantization is the main lossy step. Here, we use a limited number of reconstruction levels fewer than in the original signal. Therefore, quantization necessitates some loss of information.
- 3. Coding. Here, we assign a codeword (thus forming a binary bitstream) to each output level or symbol. This could be a fixed-length code or a variable-length code, such as Huffman coding

Shows an an audio waveform, v(t) is sampled, quantized and encoded into 3-bit PCM system

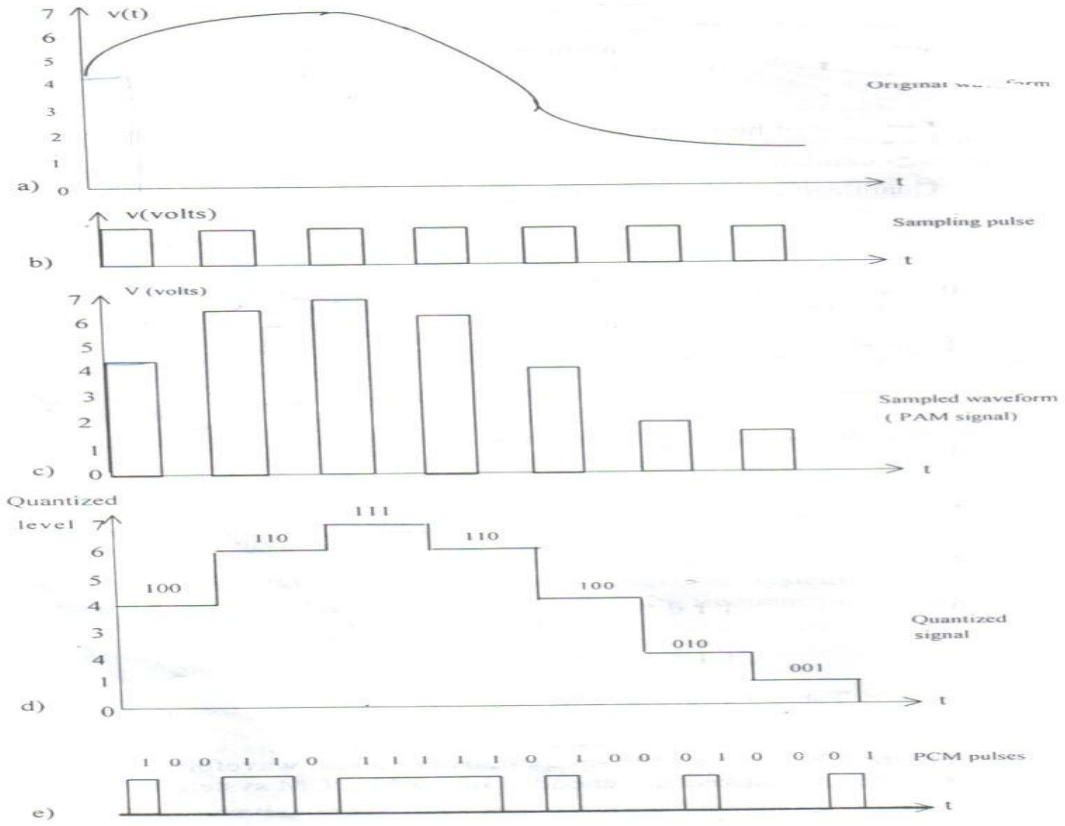


Figure 3.7 PCM: a) input signal b) sampling pulse c) sampled signal d) quantized signal e) PCM signal

PCM in general

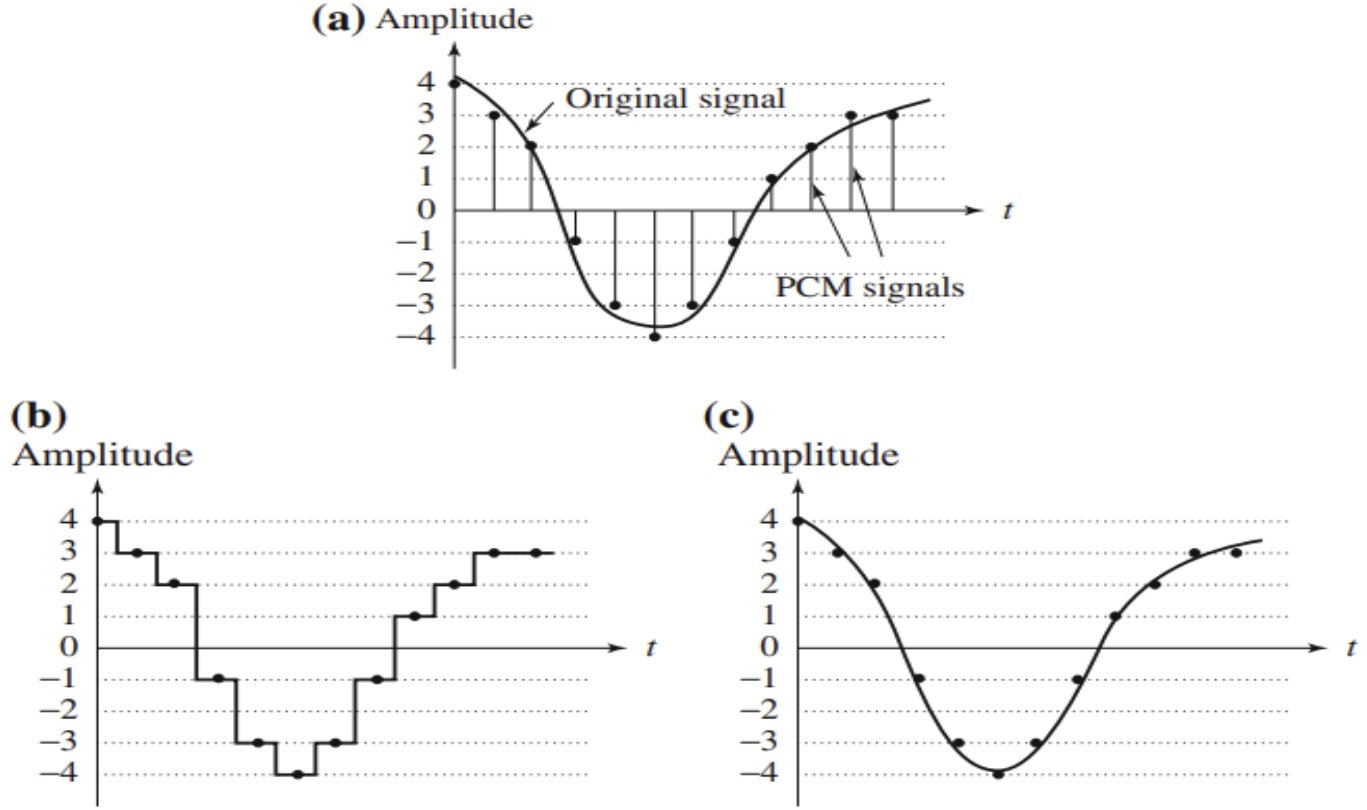


Fig. 6.14 Pulse code modulation (PCM): **a** original analog signal and its corresponding PCM signals; **b** decoded staircase signal; **c** reconstructed signal after low-pass filtering

PCM in general

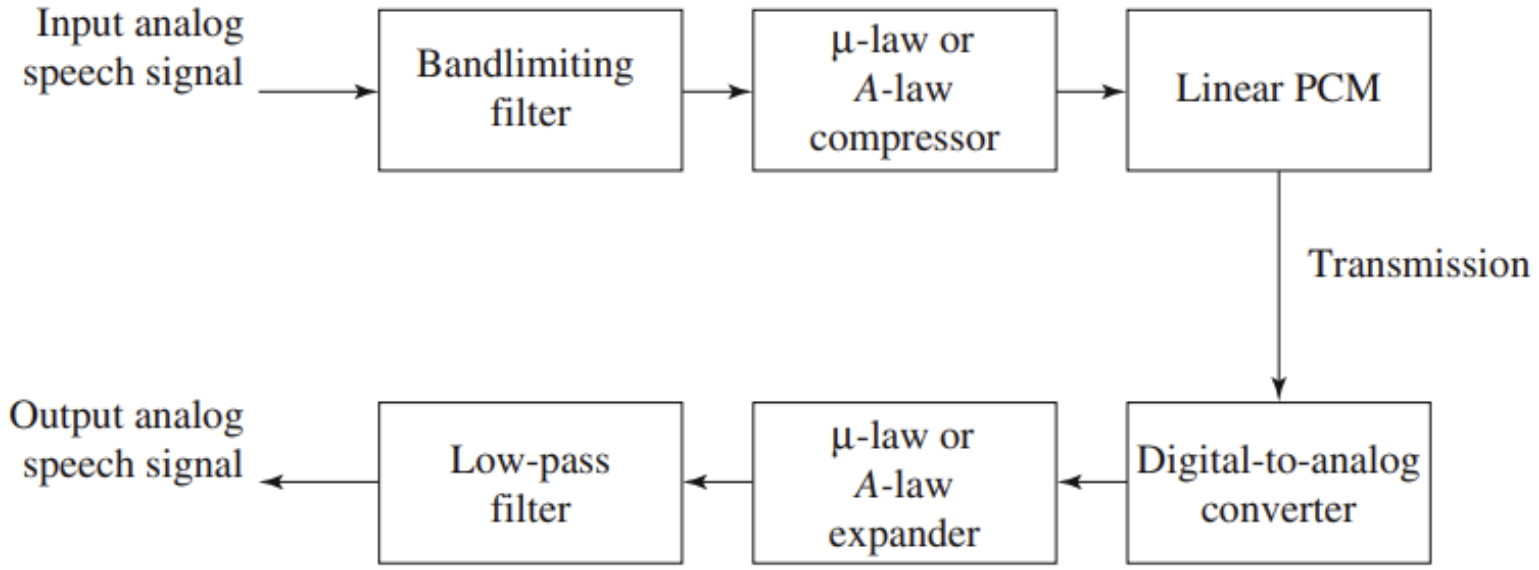


Fig. 6.15 PCM signal encoding and decoding

Compression technique

Compression: the process of coding that will effectively reduce the total number of bits needed to represent certain information.

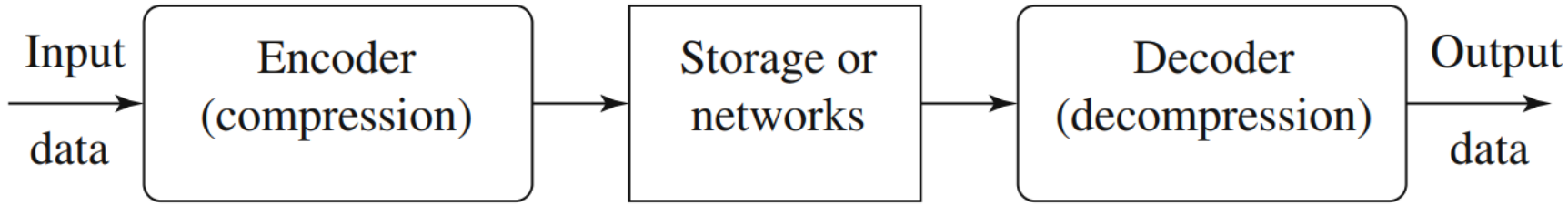


Fig. 7.1 A general data compression scheme

If the compression and decompression processes induce no information loss, then the compression scheme is **lossless**; otherwise, it is **lossy**.

Lossless compression technique

- lossless compression.
- If the total number of bits required to represent the data before compression is $B0$ and the total number of bits required to represent the data after compression is $B1$, then we define the *compression ratio* as

$$\text{compression ratio} = \frac{B_0}{B_1}.$$

we would desire any *codec* (encoder/decoder scheme) to have a *compression ratio* much larger than 1.0. The higher the *compression ratio*, the better the lossless compression scheme, as long as it is computationally feasible.

7.2 Basics of Information Theory

- The *entropy* η of an information *source* with alphabet $S = \{s_1, s_2, \ldots, s_n\}$ is:

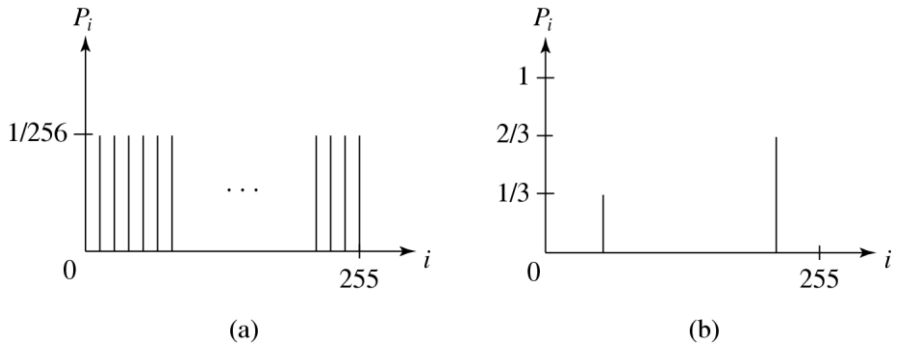
$$\eta = H(S) = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i} \qquad \text{(7.2)}$$

$$= -\sum_{i=1}^{n} p_i \log_2 p_i \qquad \text{(7.3)}$$

- p_i - probability that symbol s_i will occur in S.

- $\log_2 \frac{1}{p_i}$ - indicates the amount of information (self-information as defined by Shannon) contained in s_i, which corresponds to the number of bits needed to encode s_i.

Distribution of Gray-Level Intensities



- Fig. 7.2 Histograms for Two Gray-level Images.

- Fig. 7.2(a) shows the histogram of an image with *uniform* distribution of gray-level intensities, i.e., $\forall i\ p_i = 1/256$. Hence, the entropy of this image is:

$$\log_2 256 = 8 \qquad (7.4)$$

- Fig. 7.2(b) shows the histogram of an image with two possible values. Its entropy is 0.92.

7.3 Run-Length Coding

- **Memoryless Source**: an information source that is independently distributed. Namely, the value of the current symbol does not depend on the values of the previously appeared symbols.

- Instead of assuming memoryless source, *Run-Length Coding* (*RLC*) exploits memory present in the information source.

- **Rationale for RLC**: if the information source has the property that symbols tend to form continuous groups, then such symbol and the length of the group can be coded.

LOSSLESS COMPRESSION

- Run Length Encoding

- It removes redundancy by relying on the fact that a string contains repeated sequences or “runs” of the same symbol. The runs of the same symbol are encoded using two entities—a count suggesting the number of repeated symbols and the symbol itself. For instance, a run of five symbols

  aaaaa will be replaced by the two symbols 5a

example

BBBBEEEEEEEECCCCDAAAAA.

It can be represented as

4B8E4C1D5A.

Repetition Suppression

- Repetition suppression works by reserving a specific symbol for a commonly occurring pattern in a message. For example, a specific series of successive occurrences of the letter a in the following example is replaced by the flag ψ
- Abdcbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

is replaced by

Abdcb ψ

Pattern Substitution

- The compression is based on a simple idea that attempts to build a group of individual symbols into strings.
- The strings are given a code, or an index, which is used every time the string appears in the input message. So, the whole message consisting of symbols is broken down into groups forming substrings, each substring having an index. Its simplicity resides in the fact that no pre-analysis of the message is required to create a dictionary of strings, but rather the strings are created as the message is scanned.

Pattern Substitution

```
InputSymbol = Read Next input symbol;
String = InputSymbol;
WHILE InputSymbol exists DO
        InputSymbol = Read Next input symbol;
        IF  (String + InputSymbol) exists in DICTIONARY then
             String = String + InputSymbol
        ELSE
             Output dictionary index of String
             Add (String + InputSymbol) to DICTIONARY
             String = InputSymbol
       END // end of if statement
END // end of while loop
Output dictionary index of String
```

7.4 Variable-Length Coding (VLC)

- **Shannon-Fano Algorithm** — a top-down approach

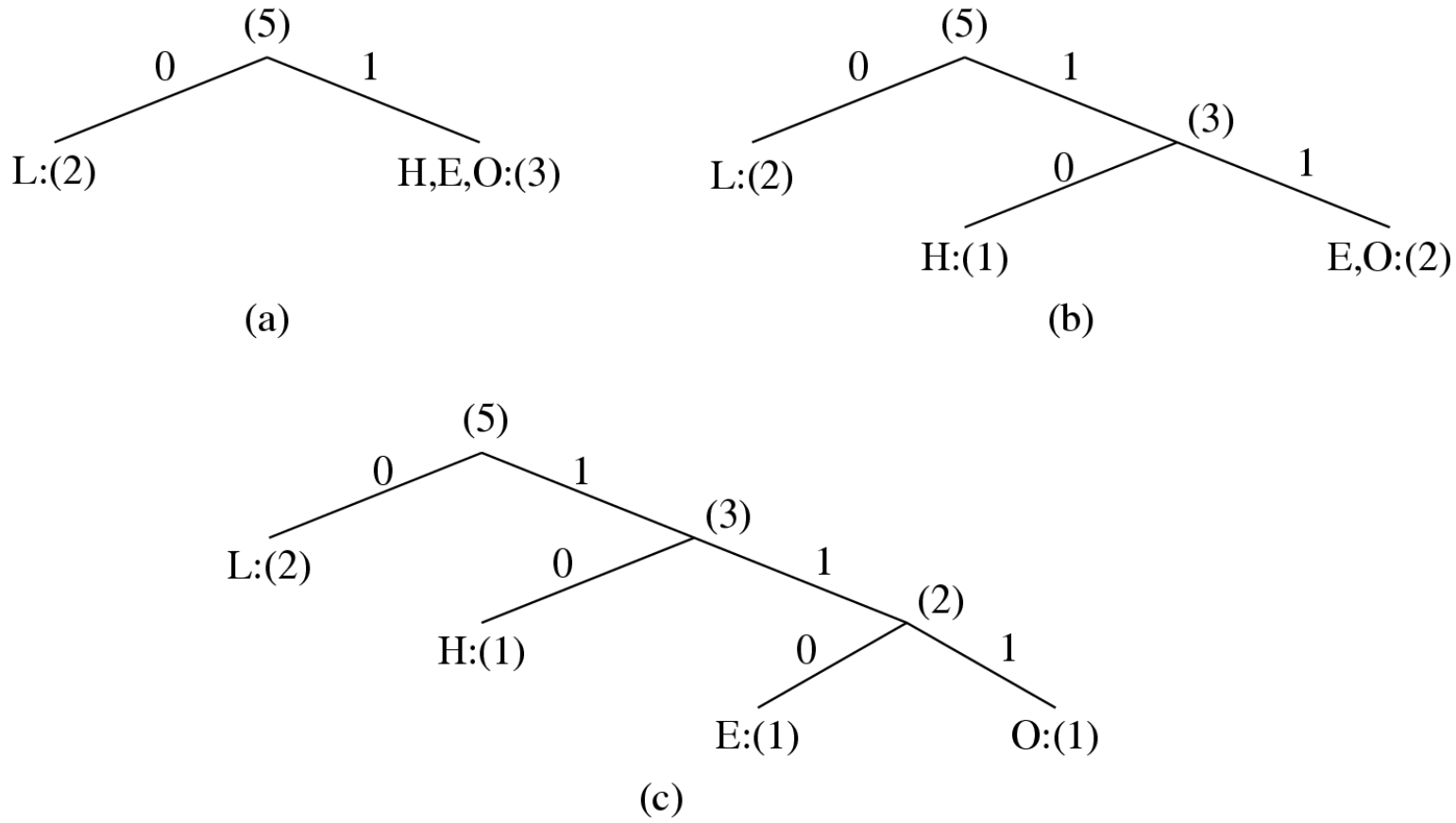
1. Sort the symbols according to the frequency count of their occurrences.

2. Recursively divide the symbols into two parts, each with approximately the same number of counts, until all parts contain only one symbol.

- **An Example: coding of “HELLO”**

Symbol	H	E	L	O
Count	1	1	2	1

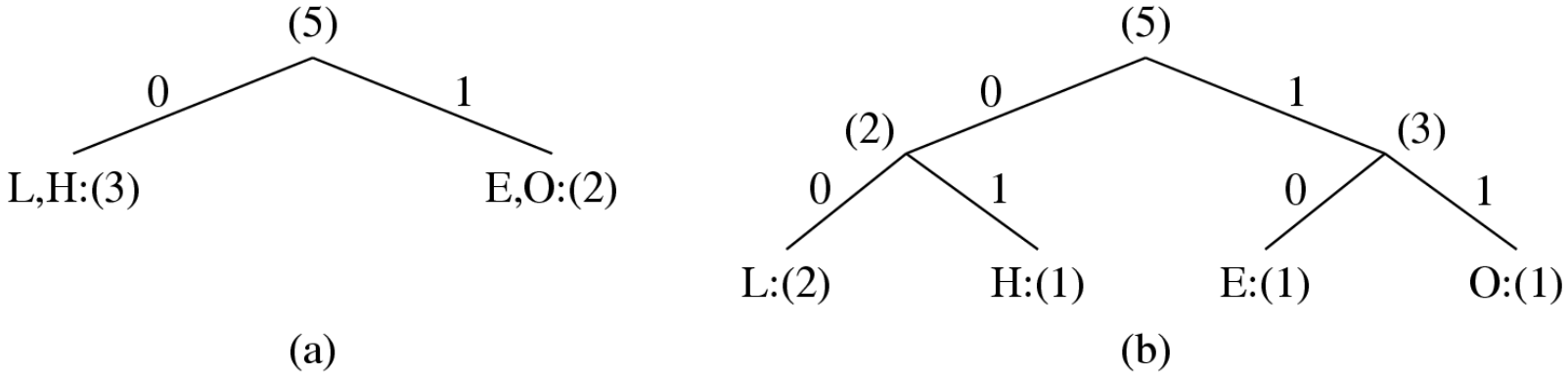
  - Frequency count of the symbols in ”HELLO”.



- Fig. 7.3: Coding Tree for HELLO by Shannon-Fano.

- **Table 7.1: Result of Performing Shannon-Fano on HELLO**

Symbol	Count	$\text{Log}_2 \frac{1}{p_i}$	Code	# of bits used
L	2	1.32	0	1
H	1	2.32	10	2
E	1	2.32	110	3
O	1	2.32	111	3
			TOTAL # of bits:	10



- Fig. 7.4 Another coding tree for HELLO by Shannon-Fano.

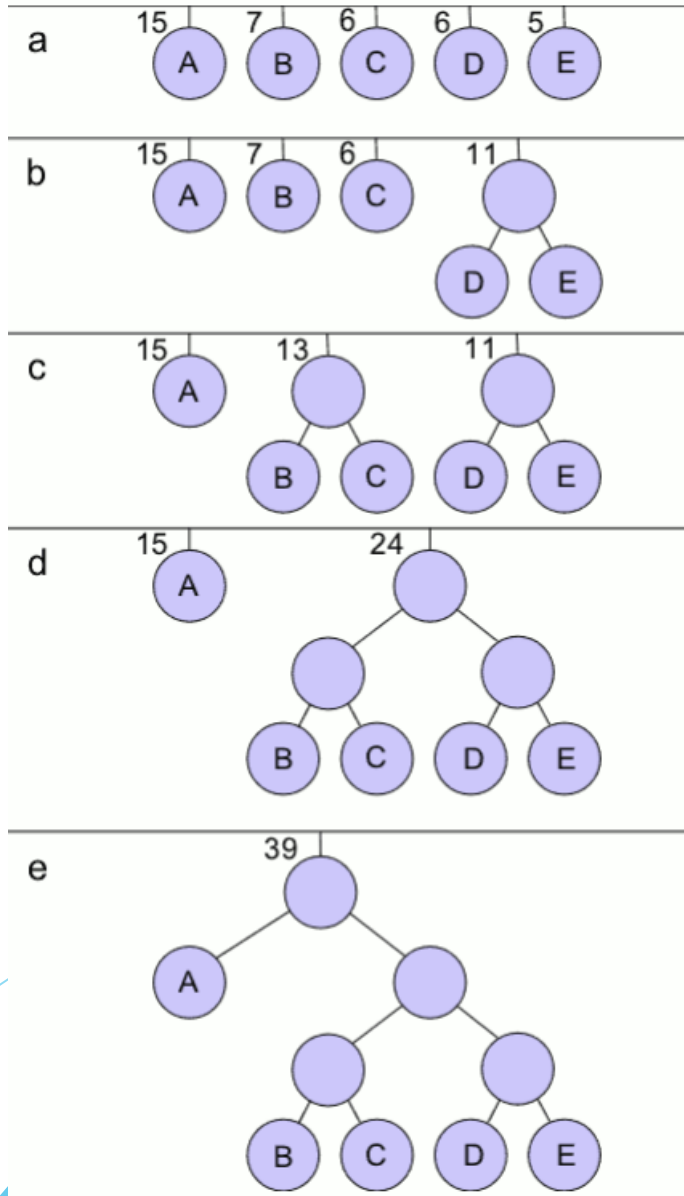
- **Table 7.2: Another Result of Performing Shannon-Fano on HELLO (see Fig. 7.4)**

Symbol	Count	$\text{Log}_2 \frac{1}{p_i}$	Code	# of bits used
L	2	1.32	00	4
H	1	2.32	01	2
E	1	2.32	10	2
O	1	2.32	11	2
			TOTAL # of bits:	10

* Huffman Coding

- When data is initially sampled, each sample is assigned the same nur $\log_2 n$f bits. This length is log2n, where n is the number or distinct samples or quantization intervals.

- Assigning an equal number of bits to all samples might not be the most efficient coding scheme because some symbols occur more commonly than others.



** Huffman Coding outline

- 1. The leaves of the binary tree are associated with the list of probabilities. The leaves can be sorted in increasing/decreasing order, though this is not necessary.
- 2. The two smallest probabilities are made sibling nodes by combining them under one new parent node with a probability equal to the sum of the child node probabilities.
- 3.Repeat the process by choosing two least probability nodes (which can be leaf nodes or new parent nodes) and combining them to form new parents until only one node is left. This forms the root node and results in a binary Huffman tree.
- 4.Label the branches with a 0 and 1 starting from the root and going to all intermediary nodes.

Huffman Coding outline

- 5 Traverse the Huffman tree from the root to a leaf node noting each branch label (1 or 0). This results in a Huffman code representation for that leaf node symbol. Perform the same for all the symbols.

Symbol	Probability	Huffman code	Code length
A	0.28	00	1
B	0.2	10	3
C	0.17	010	3
D	0.17	011	3
E	0.1	110	3
F	0.05	1110	4
G	0.02	11110	5
H	0.01	11111	5

Average symbol length = 2.63

Entropy = 2.574

Efficiency = 0.9792

Huffman Coding

ALGORITHM 7.1 Huffman Coding Algorithm— a bottom-up approach

1. Initialization: Put all symbols on a list sorted according to their frequency counts.

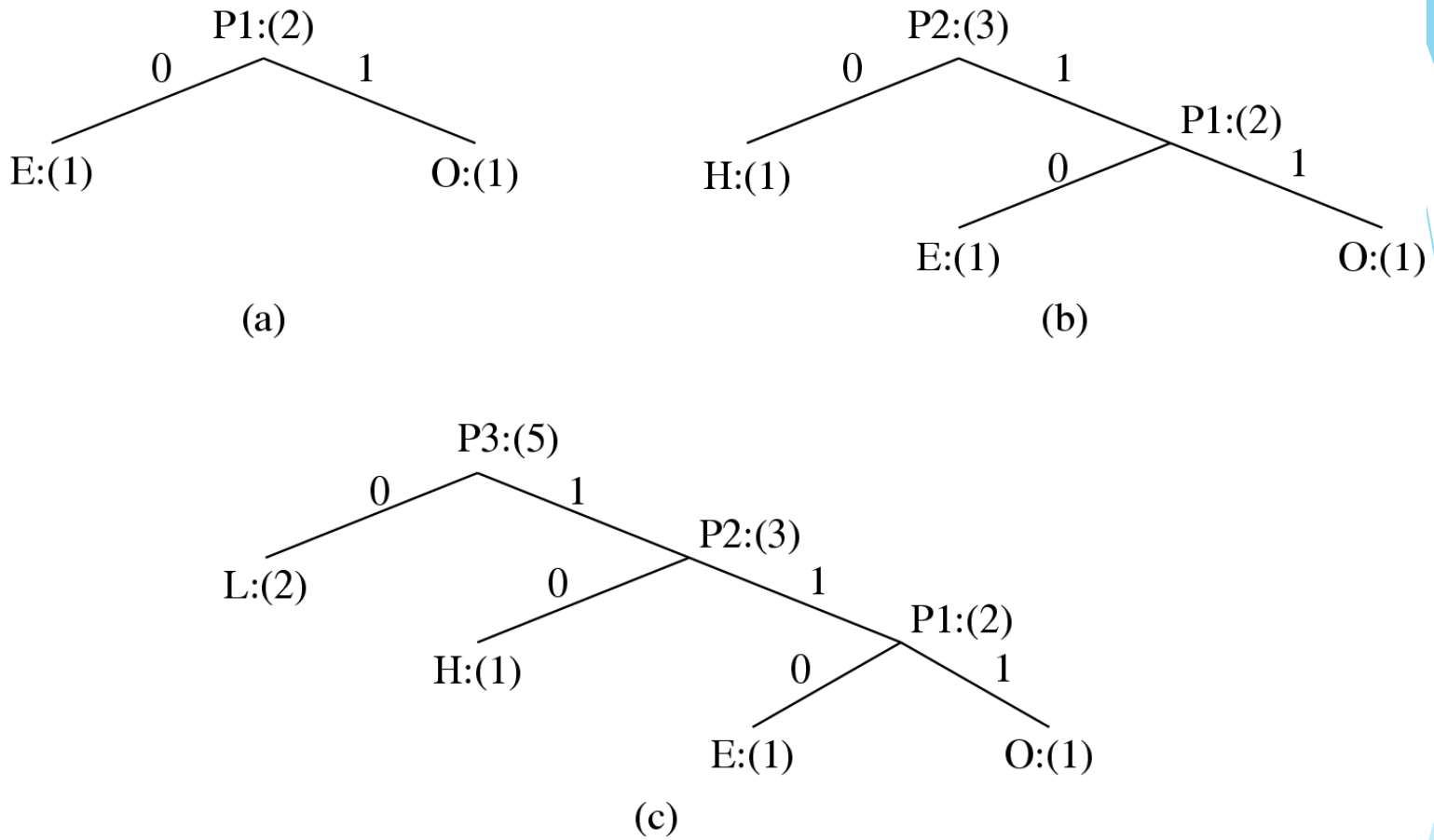
2. Repeat until the list has only one symbol left:

   (1) From the list pick two symbols with the lowest frequency counts. Form a Huffman subtree that has these two symbols as child nodes and create a parent node.

   (2) Assign the sum of the children's frequency counts to the parent and insert it into the list such that the order is maintained.

   (3) Delete the children from the list.

3. Assign a codeword for each leaf based on the path from the root.



- Fig. 7.5: Coding Tree for “HELLO” using the Huffman Algorithm.

Huffman Coding (cont'd)

- In Fig. 7.5, new symbols P1, P2, P3 are created to refer to the parent nodes in the Huffman coding tree. The contents in the list are illustrated below:

  - After initialization: L H E O
  - After iteration (a): L P1 H
  - After iteration (b): L P2
  - After iteration (c): P3

Properties of Huffman Coding

- 1. **Unique Prefix Property**: No Huffman code is a prefix of any other Huffman code - precludes any ambiguity in decoding.

- 2. **Optimality**: *minimum redundancy code* - proved optimal for a given data model (i.e., a given, accurate, probability distribution):

  - The two least frequent symbols will have the same length for their Huffman codes, differing only at the last bit.

  - Symbols that occur more frequently will have shorter Huffman codes than symbols that occur less frequently.

  - The average code length for an information source S is strictly less than $\eta + 1$. Combined with Eq. (7.5)—we have:

$$\bar{l} < \eta + 1 \qquad (7.6)$$

Extended Huffman Coding

- **Motivation**: All codewords in Huffman coding have integer bit lengths. It is wasteful when p_i is very large and hence $\log_2 \frac{1}{p_i}$ is close to 0.
- Why not group several symbols together and assign a single codeword to the group as a whole?
- **Extended Alphabet**: For alphabet $S = \{s_1, s_2, \ldots, s_n\}$, if k symbols are grouped together, then the *extended alphabet* is:

$$S^{(k)} = \{\overbrace{s_1 s_1 \ldots s_1}^{k \text{ symbols}}, s_1 s_1 \ldots s_2, \ldots, s_1 s_1 \ldots s_n, s_1 s_1 \ldots s_2 s_1, \ldots, s_n s_n \ldots s_n\}.$$

- the size of the new alphabet $S^{(k)}$ is n^k.

Extended Huffman Coding (cont'd)

- It can be proven that the average # of bits for each symbol is:

$$\eta \leq \bar{l} < \eta + \frac{1}{k} \qquad (7.7)$$

- An improvement over the original Huffman coding, but not much.

- **Problem**: If k is relatively large (e.g., $k \geq 3$), then for most practical applications where $n \gg 1$, n^k implies a huge symbol table — impractical.

Adaptive Huffman Coding

- **Adaptive Huffman Coding**: statistics are gathered and updated dynamically as the data stream arrives.

```
ENCODER                    DECODER
-------                    -------
Initial_code();            Initial_code();

while not EOF              while not EOF
 {                         {
    get(c);                      decode(c);
    encode(c);                output(c);
    update_tree(c);           update_tree(c);
 }                         }
```

-

Adaptive Huffman Coding (Cont'd)

Initial_code assigns symbols with some initially agreed upon codes, without any prior knowledge of the frequency counts.

update_tree constructs an Adaptive Huffman tree.

It basically does two things:

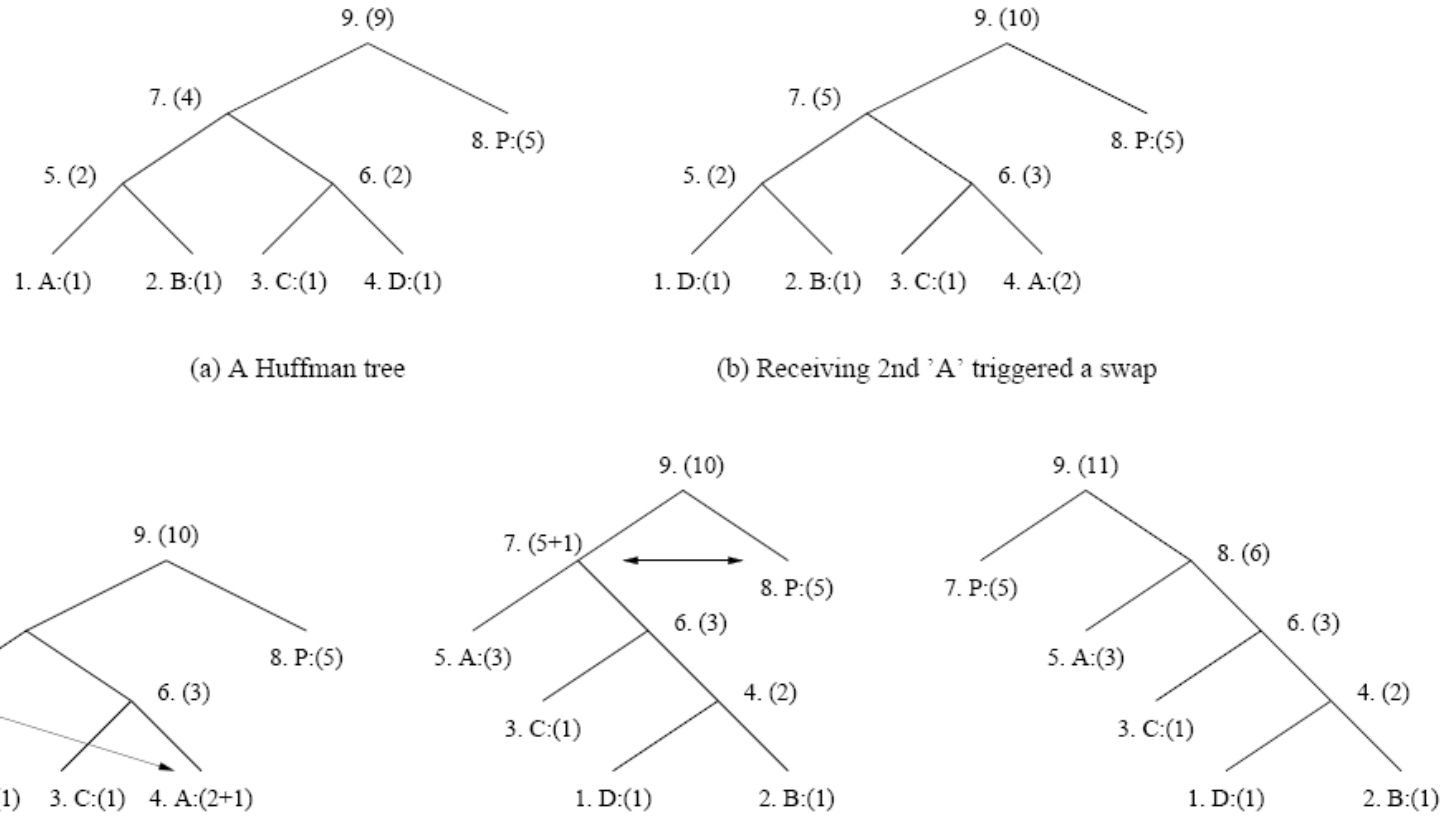
(a) increments the frequency counts for the symbols (including any new ones).

(b) updates the configuration of the tree.

The *encoder* and *decoder* must use exactly the same *initial_code* and *update_tree* routines.

Notes on Adaptive Huffman Tree Updating

- Nodes are numbered in order from left to right, bottom to top. The numbers in parentheses indicates the count.
- The tree must always maintain its *sibling* property, i.e., all nodes (internal and leaf) are arranged in the order of increasing counts.
- If the sibling property is about to be violated, a *swap* procedure is invoked to update the tree by rearranging the nodes.
- When a swap is necessary, the farthest node with count N is swapped with the node whose count has just been increased to
- N +1.



- Fig. 7.6: Node Swapping for Updating an Adaptive Huffman Tree

7.5 Dictionary-based Coding

- LZW uses fixed-length codewords to represent variable-length strings of symbols/characters that commonly occur together, e.g., words in English text.
- the LZW encoder and decoder build up the same dictionary dynamically while receiving the data.
- LZW places longer and longer repeated entries into a dictionary, and then emits the *code* for an element, rather than the string itself, if the element has already been placed in the dictionary.

Advantages of LZW over Huffman

- LZW does not require any prior knowledge of the input data stream.
- The LZW algorithm has a 60 - 70% compression ratio for some text files.
- The LZW algorithm performs better for files with a lot of repetitive data.
- The LZW algorithm compresses the data in a single pass

- ALGORITHM 7.2 - LZW Compression

```
BEGIN
    s = next input character;
    while not EOF
    {
        c = next input character;

        if s + c exists in the dictionary
                s = s + c;
        else
            {
                output the code for s;
                add string s + c to the dictionary with a new
code;
                s = c;
            }
    }
    output the code for s;
END
```

Example 7.2 LZW compression for string “ABABBABCABABBA”

- Let’s start with a very simple dictionary (“string table”), initially containing only 3 characters, with codes as follows:

Code	String
1	A
2	B
3	C

- Now if the input string is “ABABBABCABABBA”, the LZW compression algorithm works as follows:

S	C	Output	Code	String
			1	A
			2	B
			3	C
A	B	1	4	AB
B	A	2	5	BA
A	B			
AB	B	4	6	ABB
B	A			
BA	B	5	7	BAB
B	C	2	8	BC
C	A	3	9	CA
A	B			
AB	A	4	10	ABA
A	B			
AB	B			
ABB	A	6	11	ABBA
A	EOF	1		

- The output codes are: 1 2 4 5 2 3 4 6 1. Instead of sending 14 characters, only 9 codes need to be sent (compression ratio = 14/9 = 1.56).

ALGORITHM 7.3 LZW Decompression (simple version)

```
BEGIN
     s = NIL;
     while not EOF
     {
          k = next input code;
          entry = dictionary entry for k;
          output entry;
          if (s != NIL)
               add string s + entry[0] to dictionary with a new code;
          s = entry;
     }
END
```

Example 7.3: LZW decompression for string “ABABBABCABABBA”.

Input codes to the decoder are 1 2 4 5 2 3 4 6 1.

The initial string table is identical to what is used by the encoder.

The LZW decompression algorithm then works as follows:

S	K	Entry/output	Code	String
			1	A
			2	B
			3	C
NIL	1	A		
A	2	B	4	AB
B	4	AB	5	BA
AB	5	BA	6	ABB
BA	2	B	7	BAB
B	3	C	8	BC
C	4	AB	9	CA
AB	6	ABB	10	ABA
ABB	1	A	11	ABBA
A	EOF			

- Apparently, the output string is “ABABBABCABABBA”, a truly lossless result!

- **ALGORITHM 7.4 LZW Decompression (modified)**

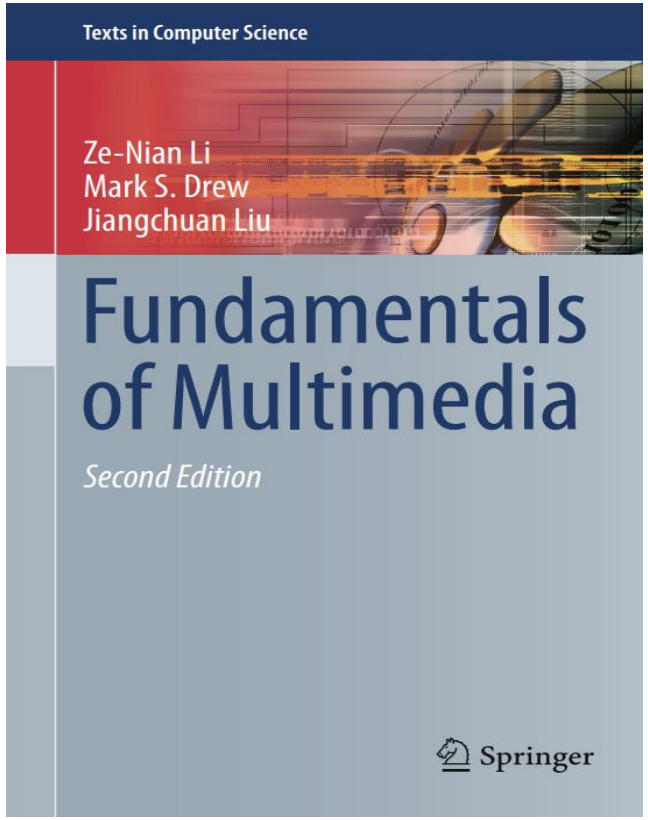
```
BEGIN
     s = NIL;
     while not EOF
     {
          k = next input code;
          entry = dictionary entry for k;

          /* exception handler */
          if (entry == NULL)
               entry = s + s[0];

          output entry;
          if (s != NIL)
               add string s + entry[0] to dictionary with a new code;
          s = entry;
     }
END
```

LZW Coding (cont'd)

- In real applications, the code length l is kept in the range of $[l_0, l_{max}]$. The dictionary initially has a size of 2^{l0}. When it is filled up, the code length will be increased by 1; this is allowed to repeat until $l = l_{max}$.

- When l_{max} is reached and the dictionary is filled up, it needs to be flushed (as in Unix *compress*, or to have the LRU (least recently used) entries removed.

Reference



Chapter 7

Question

Thank you